

**TECHNICAL SCOPE OF WORK — INTERNAL**

Deos Heartbeat Lite

Architecture, Integration & Build Plan

Not for client distribution

Audience	Thrumble engineering and delivery team
Client scope reference	Deos Heartbeat Lite, Scope of Work v2.0
Owner	Nic Harper (commercial), Guna (backend), Jo (client liaison & UAT)
Date	September 2026
Version	v1.0 — Internal Technical Scope
Build window	10 weeks from client sign off

INTERNAL DOCUMENT. Contains credentials handling guidance and implementation detail. Do not share with UKDEOS or any third party.



1. Purpose & Summary

This document is the engineering counterpart to the client facing Scope of Work v2.0 (10 week build). It defines how we build, host and maintain Deos Heartbeat Lite: report distribution management plus fleet health monitoring, delivered as one web application for a single primary user (Viv Barrett).

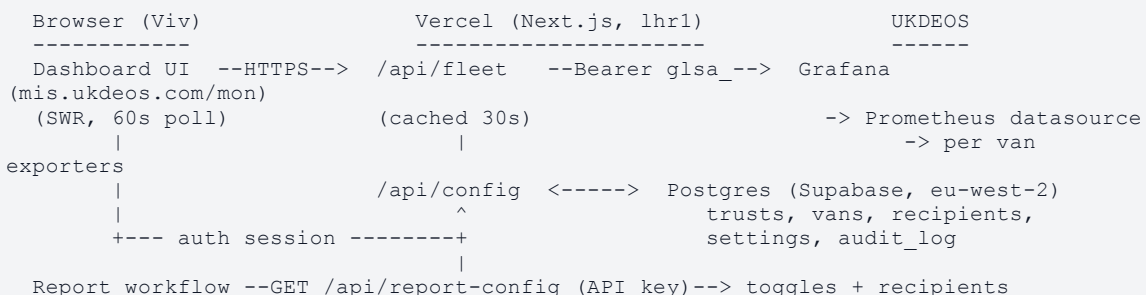
Everything in the client scope must map to something in this document. Anything described here that is not in the client scope (for example GPS polling for a fleet map) is flagged as future work and must not be built in this engagement without a change request.

1.1 What We Are Building

- A Next.js web application on Vercel, styled with shadcn/ui, behind a single secure login for Viv (positioned to the client as the foundation of the Deos Heartbeat operating system, with add on modules plugging in later)
- A configuration store (Postgres) holding trusts, vans, report toggles, email recipients and an immutable audit log
- A server side Grafana integration that proxies Prometheus queries through UKDEOS's Grafana, so the service account token never reaches the browser
- A fleet health heatmap and van detail panel driven by live metrics, polled every 60 seconds while the dashboard is open
- A change to the existing daily and weekly reporting workflows so they read their configuration from the new store instead of hard coded values

2. Architecture

2.1 High Level Flow





2.2 Stack

Layer	Choice	Notes
Frontend	Next.js (App Router), React, TypeScript	Server components for config pages; client components for live fleet views
UI	shadcn/ui + Tailwind	Matches the approved prototype. No bespoke component library
Data fetching	SWR	refreshInterval 60000, refreshWhenHidden false, revalidateOnFocus true
Hosting	Vercel Pro	Function region lhr1 (London). Preview deployments used for UAT
Database	Supabase Postgres (Pro)	Region eu-west-2 (London). Daily backups. Row level security on
ORM	Drizzle	Typed schema, migrations in repo
Auth	Supabase Auth	Single admin account for Viv, email + password with TOTP MFA. Extra users and roles are out of client scope; keep the profiles.role column so they can be added later
PDF export	Not in core scope	Trust PDFs moved to the Daily & Weekly Dashboards add on (client side react-pdf when built)
Monitoring	Sentry (free) + uptime check	Uptime check hits /api/health every 5 minutes

If we prefer to avoid Supabase, the fallback is Neon Postgres (Vercel Marketplace) plus Auth.js. Decide before Week 3 and do not mix.

3. Grafana Integration

3.1 Connection

Item	Value
Grafana base URL	https://mis.ukdeos.com/mon
Datasource	Prometheus, UID cdmwy4dypbeo0e
Query endpoint (instant)	GET /api/datasources/proxy/uid/{uid}/api/v1/query?query={promql}
Label discovery	GET /api/datasources/proxy/uid/{uid}/api/v1/label/instance/values and /label/trust/values
Auth	Authorization: Bearer {GRAFANA_TOKEN} (service account, Viewer role)
Grafana edition	Self hosted. Assumed open source. Confirm via Help menu or /api/frontend/settings buildInfo.edition



SECURITY: The current `glsa_token` has been shared in plain text in chats and documents. Before go live, ask UKDEOS to create a dedicated service account for the dashboard with the Viewer role only, issue a new token, and revoke the old one. Store the new token only in Vercel environment variables (`GRAFANA_TOKEN`). It must never be shipped to the client bundle or logged.

3.2 Metrics & Queries

Each poll runs one instant query per metric. Every query returns all vans at once, keyed by the instance and trust labels, so request volume does not grow with fleet size.

Field	PromQL	Used for
<code>modality_up</code>	<code>probe_success{job="modality"}</code>	Red status if 0. Instance label mapping to van must be confirmed (see 3.3)
<code>sync_dest_up</code>	<code>deos_sync_destination_status</code>	Red status if unreachable. Confirm value semantics (1 = reachable?)
<code>patients_today</code>	<code>orthanc_number_of_patients_today</code>	Tile + detail panel
<code>studies_today</code>	<code>orthanc_number_of_studies_today</code>	Detail panel
<code>worklist_today</code>	<code>deos_worklist_today_count</code>	Scheduled vs screened, progress bar
<code>sync_speed</code>	<code>rate(deos_sync_transfer_speed_sum[15m]) / rate(deos_sync_transfer_speed_count[15m])</code>	Average speed, MB/s. Guard divide by zero
<code>sync_failed</code>	<code>deos_sync_queue_failed_count</code>	Tile + amber rule
<code>sync_active</code>	<code>deos_sync_queue_active_count</code>	Detail panel
<code>sync_complete</code>	<code>deos_sync_queue_complete_count</code>	Detail panel
<code>sync_retry</code>	<code>deos_sync_queue_retry_count</code>	Amber rule (not displayed on tile)
<code>db_up</code>	<code>deos_db_status</code>	System health
<code>orthanc_up</code>	<code>deos_orthanc_status</code>	System health
<code>version</code>	<code>deos_version</code>	System health. Version is usually a label, not a value
<code>scrape_up</code>	<code>up</code>	Staleness detection. No data for 10 minutes = offline

Not in this engagement: `snmp_latitude`, `snmp_longitude` and `snmp_accuracy` (fleet map, future enhancement). The query layer should make adding them trivial later.



3.3 Van Identity & Mapping

- The canonical van key is the Prometheus instance label, for example bradford.van3-ingleborough. The prefix before the dot must match the trust label
- Exclude hub servers (instances ending .dserver) from the heatmap. They are not screening vans. This explains the 54 servers vs 43 vans discrepancy Viv mentioned
- The modality probe may use a different instance label (the blackbox target address). Build a mapping column vans.modality_target and populate it during discovery
- Auto discovery: on each poll, any instance present in Grafana but missing from the vans table is written as unassigned and surfaced to the admin, so new vans are never silently missed
- Display names come from the vans table, never from the raw label, so Viv can rename vans without touching Grafana

4. Polling, Caching & Load

- Client polls /api/fleet every 60 seconds via SWR, only while the tab is visible. No background polling when nobody is looking
- /api/fleet fans out the ~14 queries in parallel with a 5 second timeout each, merges results by instance, derives status, and returns one JSON payload
- Server response cached for 30 seconds (Next.js revalidate) so multiple tabs or users share one set of Grafana calls
- If Grafana fails or times out, return the last good payload with stale: true and a timestamp. The UI shows a 'Data delayed' banner rather than blanking the heatmap
- No time series is stored in this engagement. Fleet health is current day only, which keeps the database tiny. Storing history is part of the future alerting and AI work

4.1 Expected Volume

Scenario	Grafana queries	Vercel function calls / month
Viv has dashboard open 8 hrs every weekday	~14 per minute while open	~10,000 to 15,000
Dashboard open 24/7 on a wall screen	~14 per minute	~45,000
Vercel Pro included allowance	n/a	1,000,000 (confirm current figure)

Load on UKDEOS's Grafana is negligible at either level. Grafana API calls are not billed (self hosted).



5. Status Derivation Rules

Status is computed server side in one pure function (`deriveStatus`) with unit tests. Thresholds live in the settings table so they can be tuned without a deploy.

Status	Rule (first match wins)	Default threshold
Red: Offline	<code>modality_up = 0 OR sync_dest_up = 0 OR no scrape data for 10 minutes</code>	10 min staleness
Grey: Not scheduled	<code>worklist_today = 0 AND patients_today = 0</code>	n/a
Amber: Degraded	<code>sync_failed >= threshold OR sync_retry >= threshold OR sync_speed below floor for 15 min OR after 12:00 screened/scheduled below threshold</code>	failed 3, retry 5, speed 0.25 MB/s, progress 40%
Green: Online	None of the above	n/a

The grey state avoids a common false alarm: vans with no appointments today (for example Derby van4, London van3) should not show red. Confirm the rule with Viv in the UI sessions.

Known low baseline vans (Reading perky at 0.14 MB/s, Swindon rio at 0.22 MB/s) will trip the speed floor permanently. Support a per van override (`vans.speed_floor`) rather than lowering the fleet wide floor.

6. Data Model

```
trusts      (id, name, slug, daily_enabled bool, weekly_enabled bool,
             active bool, created_at)
vans        (id, trust_id -> trusts, instance text unique, display_name,
             modality_target,
             daily_enabled bool, weekly_enabled bool, speed_floor numeric null,
             status enum(active, paused, unassigned, removed), created_at)
recipients  (id, trust_id -> trusts, name, email, active bool, created_at)
settings    (key text pk, value jsonb)          -- thresholds, poll interval
users       (Supabase auth) + profiles (user_id, role enum(admin, viewer))
report_runs (id, trust_id, report_type, run_at, status, error)
audit_log   (id, at timestamptz, user_id, action, entity, entity_id,
             before jsonb, after jsonb)        -- insert only, no update/delete grants
```

- Deletes in the UI are soft deletes (`status = removed` or `active = false`). Nothing is hard deleted, which keeps the audit trail meaningful
- `audit_log` is written by a Postgres trigger on `trusts`, `vans` and `recipients` so the log cannot be bypassed by application bugs
- RLS: admin can write everything; viewer is read only; the reporting workflow uses a separate API key with read only access to `/api/report-config`



7. Reporting Workflow Integration

The existing daily (7 to 8am) and weekly reports keep their generation logic. Only their configuration source changes.

- New endpoint GET `/api/report-config` returns active trusts, their vans with daily/weekly flags, and recipient emails. Authenticated with a static API key (`REPORT_CONFIG_KEY`) held in the workflow's credentials
- Workflow change: replace the hard coded trust, van and recipient lists with one HTTP call at the start of each run, then filter by the flags
- Paused or removed vans are skipped; trusts with no active recipients are skipped and logged
- Run status: at the end of each run the workflow posts the result per trust to POST `/api/report-runs` (same API key). Stored in `report_runs` (id, trust_id, report_type, run_at, status, error). Drives the green/amber/red report status indicator (red if the last run for that trust failed). This was in Aaron's v1.1 scope and is carried into v2.0
- Fallback: if the endpoint is unreachable, the workflow uses the last successful config snapshot and alerts the Thrumble channel, so reports never silently stop
- The Wednesday Email Tracking Automation is out of scope and must not be touched

Confirm during Week 1 which tool runs the current reports (assumed n8n) and where their lists currently live, then plan the migration of existing trust, van and recipient data into Postgres.

8. Security & Data Protection

- No patient identifiable data is processed or stored. The platform handles aggregate counts, device status and staff email addresses only. Record this in the handover notes, as NHS trusts may ask
- UK data residency: Vercel functions pinned to `lhr1`, Supabase in `eu-west-2`
- Secrets only in Vercel environment variables: `GRAFANA_TOKEN`, `SUPABASE` keys, `REPORT_CONFIG_KEY`. Separate values for preview and production
- Admin account uses MFA. Session timeout 12 hours
- Security headers (CSP, HSTS) set in `next.config`. Rate limit `/api/report-config` and auth routes
- Access for Thrumble support via a named viewer or admin account, never by sharing Viv's login



9. Environments, Testing & Deployment

Environment	Purpose	Data
Local	Development	Seeded fixtures captured from real Grafana CSV exports
Preview (Vercel)	Per branch review and client UAT (Weeks 9 to 10)	Staging Supabase project, live Grafana (read only)
Production	Live platform	Production Supabase project, live Grafana

- Unit tests: deriveStatus, metric merging by instance, report config filtering
- Integration test: /api/fleet against recorded Grafana responses, including timeout and empty result cases
- Manual QA checklist before UAT covering every client scope bullet
- /api/health checks database and Grafana reachability and token validity. Uptime monitor alerts Thrumble if it fails, which catches an expired or revoked token before Viv notices

10. Build Plan & Ownership

Aligned to the 10 week client timeline. Guna owns backend and integration; Jo owns client liaison, data collection and UAT coordination. Frontend owner to be confirmed.

Weeks	Engineering tasks	Owner
1 to 2	Repo, Vercel and Supabase setup. Confirm Grafana edition, labels and value semantics. Build instance and modality mapping. Audit current report workflow. Two UI sessions with Viv; mockups signed off	Guna, Jo (sessions, data)
3 to 4	Schema, migrations, audit trigger, RLS. Migrate existing trusts, vans and recipients. /api/report-config. Update reporting workflow and run in parallel with old config for one cycle	Guna
5 to 6	Auth and roles. Report management UI: trusts, vans, toggles, recipients, audit log view	Frontend (TBC), Guna
7 to 8	/api/fleet, caching, stale handling, deriveStatus with tests. Heatmap, trust collapse, trust filter (per trust dashboard, stats recalculated client side from the same payload), van detail panel	Guna, Frontend (TBC)
9	Internal QA against checklist. Deploy to preview. Viv UAT with test script	Jo (UAT), Guna
10	Fixes. Rotate to new Grafana token. Production deploy. Walkthrough with Viv. Handover notes	Guna, Jo



11. Infrastructure & Maintenance

11.1 Infrastructure

Item	Plan	Notes
Vercel	Pro	Per seat, not per project. Use the existing Thrumble seat if available
Supabase	Pro	Postgres, auth, daily backups. Neon free tier is the fallback option
Grafana API	Existing UKDEOS instance	Self hosted by UKDEOS, no metering
Sentry, uptime monitor	Free tiers	Sufficient for one user
Domain	ukdeos.com subdomain	For example dashboard.ukdeos.com, DNS managed by UKDEOS

11.2 Maintenance Effort

Budget 2 to 4 engineering hours per month for token rotation, dependency and security updates, adapting to Grafana metric or label changes, onboarding new trusts if auto discovery needs help, and minor requests from Viv.



12. Open Questions to Resolve in Week 1

#	Question	Who
1	Grafana edition and version. Can UKDEOS create a Viewer service account?	Jo to ask Viv (or whoever manages UKDEOS Grafana)
2	Value semantics of deos_sync_destination_status and the deos_* status gauges (1 = OK?)	Guna, verify in Grafana
3	How probe_success{job="modality"} instances map to van instances	Guna
4	Which tool runs the existing daily and weekly reports, and where the lists live	Guna
5	Exact active van list (exclude dservers, xray-mob units?) and TIC entry in the data	Jo with Viv
6	Grey 'not scheduled' state and amber thresholds acceptable to Viv	Jo in UI sessions
7	Preferred login method and whether any read only users are needed at launch	Jo with Viv
8	Subdomain on ukdeos.com and who manages their DNS	Jo with Viv

13. Explicitly Out of This Build

- Optional add ons (Section 14): Live Satellite Fleet Map and 24 Hour Van History upgrade, Daily & Weekly Dashboards, Freshdesk Tickets, Email Tracking, and Trust Portal (discovery first). Build only if ordered
- Alerting, notifications, background 24/7 polling and Jo voice agent integration (future)
- Historical storage and trends unless the Daily & Weekly Dashboards add on is ordered. Anomaly detection and AI copilot (future)
- Any Grafana side changes, dashboards or exporters
- Client scope Section 8.1 excludes liability for Grafana and third party outages, data issues, and changes to Grafana software, config, metrics, labels or API. Any rework caused by such changes is a paid change request; flag it to Nic before starting work
- Changes to report templates. The Wednesday Email Tracking Automation is untouched unless the Email Tracking add on is ordered
- Multiple users, roles and permissions (single admin login only). Trust PDF export (Daily & Weekly Dashboards add on)

Design the query layer and data model so these can be added without a rebuild, but do not build them now. Anything beyond the client scope needs a change request signed by Viv.



14. Optional Add On Build Notes

Only build if ordered. Estimates assume the core platform is complete; shared components are reused.

14.1 Live Satellite Fleet Map and 24 Hour Van History upgrade

- Map: Mapbox GL JS with the satellite streets style (recommended). Alternatives: Google Maps JS (hybrid) or MapLibre with MapTiler satellite. One user stays within free tiers; confirm current limits before build
- Add `snmp_latitude`, `snmp_longitude`, `snmp_accuracy` to `/api/fleet`. Treat 0,0 as no fix; accuracy of 10 m or worse as low accuracy. Observed spread in the Grafana export: mostly 0.4 to 0.7 m, London van7 10.1 m, Plymouth mamo 500 m, Derby van2 no fix
- Show `snmp_accuracy` (metres) and last position timestamp in the marker tooltip and van panel.
- Render an accuracy circle (radius = `snmp_accuracy`, metres) for positions at or above the low accuracy threshold; do not plot no fix vans. 24 hour trails should skip or smooth low accuracy points to avoid false jumps
- Status coloured markers, clustering at national zoom, trust and status filters, click through to van detail panel, GPS health panel
- 24 hour history: `query_range` per van over 24h (step 1 to 5 min depending on GPS scrape interval), polyline trail, time slider
- Estimate: 4 to 5 days live map; 7 to 9 days with history

14.2 Daily & Weekly Dashboards

- Own history store (client facing commitment): nightly Vercel Cron job (00:15) writes one row per van per day to `van_daily`: `van_id`, `date`, `scheduled`, `patients`, `studies`, `avg_upload_time`, `avg_sync_speed`, `uptime_pct`, `sync_failed_total`, `minutes_offline`. Views read `van_daily`, never Prometheus directly, so they survive any Grafana retention change
- Daily values from Prometheus at end of day:
`max_over_time(orthanc_number_of_patients_today[1d])` and equivalents;
`avg_over_time(probe_success{job="modality"}[1d])` for uptime; rate based averages for `deos_sync_transfer_time` (upload time) and `deos_sync_transfer_speed`
- Backfill on launch via `query_range` for whatever Prometheus still holds (default 15 days). Daily view compares with the same day last week, so meaningful comparisons need 8+ days of history
- Daily / Weekly toggle on every view. Trust dashboards aggregate across vans: sum scheduled, patients, studies, failures, offline minutes; average upload time, sync speed and uptime (weight upload time and speed by transfer count, not a simple mean of van averages). Van dashboards show single van values



- Fleet overview with trust league table; trust dashboard with daily chart and underlying van table linking to van pages; charts via shadcn charts (Recharts). Daily or weekly trust PDF export (client side react-pdf)
- Cache historical responses for 15 minutes
- Estimate: about 8.5 days (van_daily job and backfill 2 to 2.5, van page 1.5 to 2, trust page with aggregation 2, fleet overview 1, PDF 0.5 to 1, testing 1)

14.3 Freshdesk Tickets: Open Tickets & History (requires Daily & Weekly Dashboards)

- Freshdesk REST API with a UKDEOS API key. Sync every 15 minutes into a tickets table (id, trust_id, van_id, status, priority, subject, requester, source, agent, created_at, updated_at, resolved_at) plus ticket_activity for the timeline. Respect Freshdesk plan rate limits
- Match tickets to vans using the trust and van custom fields Jo already populates. Unmatched tickets land as unassigned for Viv to link
- Surfaces: Tickets page (open list, counts, trust and status filters); ticket count badge on heatmap tiles; open tickets in the live van panel; ticket detail view with activity timeline, deep link to the Freshdesk ticket URL, and link to the van; history on van and trust daily and weekly dashboards
- Read only. No replies, status changes or ticket creation from Deos Heartbeat (would require two way sync and conflict handling)
- Estimate: 5.5 to 6.5 days (was 4 to 5 before the open tickets page and ticket detail were added)

14.4 Trust Portal (discovery first)

- Client document presents this as discovery only: trust needs research, compliance and IG review (DPIA, DSPT, Cyber Essentials, DTAC, clinical safety applicability, NHS sign in), commercial and support model. Scope only after discovery. The notes below are an initial technical view to inform discovery, not a committed design
- Multi tenant access: trust_users table linking auth users to a single trust. Enforce isolation with Postgres RLS on every table and server side filtering on /api/fleet, weekly and map endpoints. Never filter trust data client side only
- Per trust cache keys so trust users share Grafana calls without seeing other trusts' data
- Admin features for Viv: invite and remove trust users, enable or disable each trust's portal, view trust login history
- MFA mandatory for trust users. Session and login audit logged
- Security testing before pilot: attempt cross trust access via IDs, URLs and API calls; document results
- Delivered after core go live: build and test, then pilot with one trust, then roll out
- Initial build estimate before discovery: 11 to 14 days. Excludes compliance work such as Cyber Essentials Plus certification, external penetration testing, DTAC evidence, or NHS sign in integration, any of which discovery may make mandatory



14.5 Email Tracking in Deos Heartbeat

- First confirm how the existing Wednesday Email Tracking automation works, where it runs, and what it outputs. The estimate assumes it can expose its output and read its on/off flag and recipients from the config store
- Add the report to the report management UI with the same toggle and recipient controls; store each week's output (or a link to it) for viewing history in the dashboard; log changes to audit_log
- Estimate: 3 to 4 days. Re-estimate and flag to Nic if the automation needs significant rework to integrate

Complete Visibility bundle: Fleet Map, 24 Hour Van History upgrade, and Daily & Weekly Dashboards. Add ons confirmed by end of discovery are built alongside core; most fit within 10 weeks, bundle up to 2 extra weeks. Van dashboards embed the 24 hour trail map, so build the fleet map first when both are ordered.

— END OF INTERNAL TECHNICAL SCOPE —